

ML Experiment Tracking with MLflow

House Price Prediction on the California Housing Dataset

Project Documentation

1. Introduction

This document describes a machine learning experiment tracking project developed as a task of the Samsung Innovation Campus program. The project uses MLflow to track and compare multiple training runs of the same regression model on a house price prediction problem.

The same model is trained three times with different hyperparameters. For each run, MLflow records the parameters, the validation metrics, and the trained model as an artifact. The recorded results are then compared in order to select the best-performing model. The workflow followed in this project is: Train, Track, Compare, and Select.

2. Objectives

- Train a regression model for house price prediction using the California Housing dataset.
- Train the same model with three different hyperparameter configurations.
- Track the parameters, metrics, and trained model of every run using MLflow.
- Compare the runs in the MLflow user interface.
- Select the best-performing model using the Root Mean Squared Error (RMSE) as the primary metric.

3. Dataset and Data Preparation

3.1 Dataset

The project uses the California Housing dataset, which is loaded through the scikit-learn library. The target variable is the median house value, expressed in units of \$100,000.

3.2 Data Splitting

The dataset is divided into a training set and a validation set:

- 80% of the data is used for training (16,512 samples).
- 20% of the data is used for validation (4,128 samples).

A fixed random seed (`random_state = 42`) is used for the split, which makes the split reproducible and ensures that all runs are evaluated on the same validation data.

4. Technologies and Tools

- **Python:** the programming language used to implement the project.

- **MLflow:** used for experiment tracking, model logging, and result comparison.
- **scikit-learn:** used for loading the dataset, splitting the data, training the model, and computing the evaluation metrics.
- **pandas and NumPy:** used for data handling and numerical computation.
- **SQLite:** used as the MLflow tracking backend (mlflow.db).
- **Git and GitHub:** used for version control and for delivering the project repository.

5. Model and Experiment Configuration

5.1 Model

The model used in all experiments is the GradientBoostingRegressor from scikit-learn. This model was selected because it supports both the `max_depth` and `learning_rate` hyperparameters required by the task.

5.2 Experiment Configurations

Three runs were executed. Only the `max_depth` and `learning_rate` hyperparameters differ between them:

- **Run 1:** `max_depth = 3`, `learning_rate = 0.1`
- **Run 2:** `max_depth = 5`, `learning_rate = 0.05`
- **Run 3:** `max_depth = 7`, `learning_rate = 0.01`

The number of estimators was fixed at 100 for all runs (`n_estimators = 100`), and the random seed was fixed at 42. The number of estimators is also logged as an additional parameter in MLflow.

5.3 Evaluation Metrics

The following metrics are calculated on the validation set:

- **RMSE:** the square root of the mean squared error. Lower values indicate better performance. This is the primary metric for model selection.
- **MAE:** the mean absolute error. Lower values indicate better performance.
- **R²:** the coefficient of determination. Higher values indicate better performance.

6. Implementation

6.1 Project Structure

The project repository contains the following files:

- **train.py:** loads the dataset, splits the data, trains the three experiments, and logs the parameters, metrics, and models using MLflow.
- **requirements.txt:** lists the Python dependencies of the project.

- **README.md:** describes the task, how to run the project, the results, and the selected model.
- **screenshots:** contains the screenshots of the MLflow experiment results.
- **results.csv:** contains the comparison of the three runs generated by the training script.

6.2 Tracking Procedure

For every run, the training script performs the following steps within an MLflow run context:

1. Train the model using the hyperparameters of the run.
2. Calculate the RMSE, MAE, and R^2 metrics on the validation set.
3. Log the parameters `max_depth`, `learning_rate`, and `n_estimators`.
4. Log the metrics `rmse`, `mae`, and `r2`.
5. Save the trained model as an MLflow artifact, together with its input and output signature and an input example.

After the three runs are completed, the script queries MLflow for the runs of the experiment, sorts them by RMSE in ascending order, prints the comparison, identifies the best run, and saves the comparison to `results.csv`.

6.3 Running the Project

The project is executed using the following steps:

6. Create and activate a Python virtual environment.
7. Install the dependencies using the command: `pip install -r requirements.txt`
8. Run the experiments using the command: `python train.py`
9. Open the MLflow user interface using the command: `mlflow ui --backend-store-uri sqlite:///mlflow.db`
10. Open `http://localhost:5000`, select the experiment `California-House-Price-Prediction`, select the three runs, and choose Compare.

7. Results

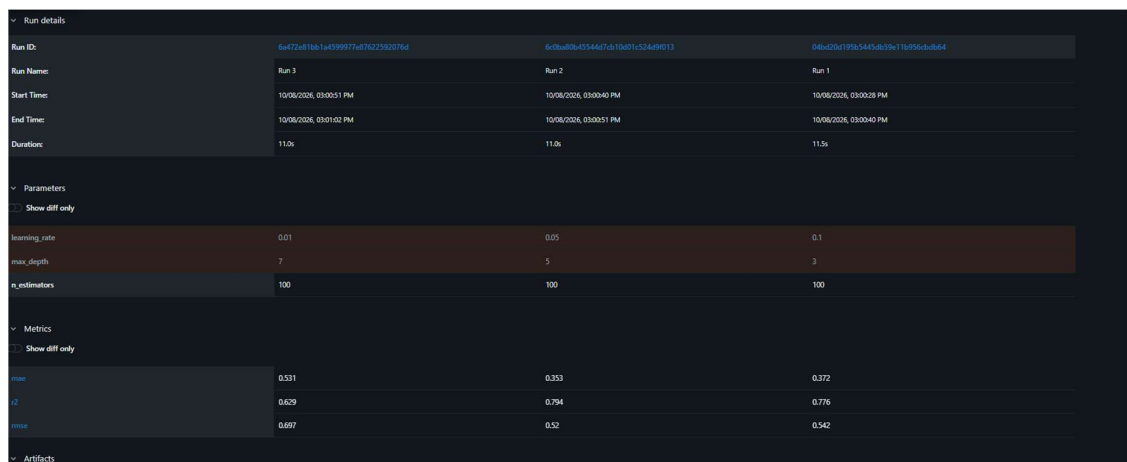
7.1 Run Results

The validation results of the three runs are as follows:

- **Run 1** (`max_depth = 3`, `learning_rate = 0.1`): RMSE = 0.5422, MAE = 0.3716, $R^2 = 0.7756$.
- **Run 2** (`max_depth = 5`, `learning_rate = 0.05`): RMSE = 0.5198, MAE = 0.3531, $R^2 = 0.7938$.
- **Run 3** (`max_depth = 7`, `learning_rate = 0.01`): RMSE = 0.6973, MAE = 0.5311, $R^2 = 0.6290$.

7.2 MLflow Comparison

Figure 1 shows the comparison of the three runs in the MLflow user interface. MLflow lists the most recent run first; therefore, the columns appear in the order Run 3, Run 2, and Run 1.



Run ID	6a172a81b61a4539977a87622592076d	6c0ba80b4554427cd10d71c534a9d13	04ba02d195b5445db59e11b955cdu8d4
Run Name	Run 3	Run 2	Run 1
Start Time	10/06/2026, 03:00:51 PM	10/06/2026, 03:00:40 PM	10/06/2026, 03:00:28 PM
End Time	10/06/2026, 03:01:02 PM	10/06/2026, 03:00:51 PM	10/06/2026, 03:00:40 PM
Durations	11.0s	11.0s	11.5s
Parameters			
learning_rate	0.01	0.05	0.1
max_depth	7	5	3
n_estimators	100	100	100
Metrics			
rmse	0.531	0.353	0.372
r2	0.629	0.794	0.776
auc	0.697	0.52	0.542
Artifacts			

Figure 1. Comparison of the run details, parameters, and metrics in the MLflow user interface.

Figure 2 shows the parallel coordinates plot generated by MLflow, which relates the learning rate and the maximum depth to the RMSE of each run.

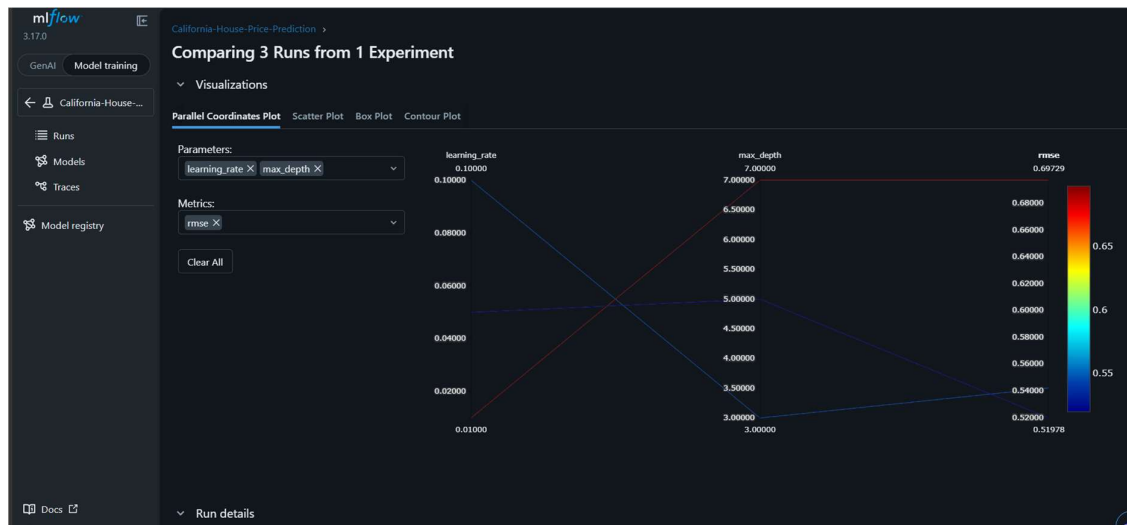


Figure 2. Parallel coordinates plot of learning_rate and max_depth against RMSE.

8. Model Selection

8.1 Selected Model

Run 2 is selected as the best model, with max_depth = 5, learning_rate = 0.05, and an RMSE of 0.5198.

8.2 Justification

RMSE is the primary selection metric, and Run 2 achieved the lowest RMSE among the three runs. Run 2 also achieved the lowest MAE and the highest R^2 , so the selection is consistent across all three metrics.

- **Run 1** obtained a higher RMSE than Run 2 (0.5422 compared with 0.5198).
- **Run 3** obtained the highest RMSE (0.6973) and the lowest R^2 (0.6290). A possible explanation is that the very small learning rate (0.01), combined with only 100 estimators, does not allow the boosting process to converge sufficiently, despite the greater tree depth. This explanation was not verified by additional experiments.

Because the target is expressed in units of \$100,000, the RMSE of Run 2 corresponds to an average prediction error of approximately \$52,000.

9. Challenges and Solutions

9.1 Deprecated File-Based Tracking Backend

The installed MLflow version (3.17.0) rejected the file-based tracking backend (`./mlruns`), which is in maintenance mode, and raised an exception when the experiment was created. The tracking URI was changed to a SQLite database (`sqlite:///mlflow.db`), and the MLflow user interface is started with the matching `--backend-store-uri` option.

9.2 Untrusted Type Error During Model Logging

When logging the model, MLflow serialized the scikit-learn model using the `skops` library and raised an exception because the type `sklearn.tree._tree.Tree`, which is used internally by the `GradientBoostingRegressor`, was not in the list of trusted types. Since the models are trained within the project itself, this type was explicitly declared as trusted by passing it to the `skops_trusted_types` parameter of the model logging function.

9.3 Model Logging Parameter Name

In MLflow 3, the `name` argument is used to name the logged model instead of the `artifact_path` argument. The training script was updated accordingly.

10. Limitations

- Only three hyperparameter configurations were evaluated, in which `max_depth` and `learning_rate` vary together.
- The number of estimators was fixed at 100 for all runs.
- The models were evaluated on a single validation split; cross-validation was not performed.
- The generated `mlflow.db` file and the `mlruns` directory are excluded from the repository; the results are therefore documented through screenshots, the comparison table, and `results.csv`.

11. Conclusion

This project demonstrated the Train, Track, Compare, and Select workflow using MLflow. A GradientBoostingRegressor was trained three times on the California Housing dataset with different values of max_depth and learning_rate. The parameters, metrics, and models of all runs were tracked in MLflow and compared in its user interface. Using RMSE as the primary metric, Run 2 (max_depth = 5, learning_rate = 0.05) was selected as the best model, with an RMSE of 0.5198, an MAE of 0.3531, and an R^2 of 0.7938.